easyep.py — technical

`src/moe_pruning/scoring/easyep.py · 96 lines · moe_pruning.scoring.easyep`

Purpose

EASY-EP quantities and selection rules.

Formulas

```
s_t = max(1 - cos(h_t, h_t + y_t), 0) (h = residual entering the MoE sub-layer, y = MoE output)
c_{e,t} = g_{e,t} * ||E_e(x_t)||_2; I_e = sum_t c_{e,t} * s_t
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`token_contribution`

```
def token_contribution(residual: torch.Tensor, moe_output: torch.Tensor) -> torch.Tensor
```

lines 53-57

s_t = max(1 - cos(h_t, h_t + y_t), 0), computed in float32. Shapes [..., D] -> [...].

1. Compute in float32 with `F.cosine_similarity` over the last dimension; clamp at 0.

`output_aware_importance`

```
def output_aware_importance(gate: torch.Tensor, expert_output: torch.Tensor) -> torch.Tensor
```

lines 60-62

c_{e,t} = g_{e,t} * ||E_e(x_t)||_2. Shapes gate [...], expert_output [..., D] -> [...].

`rank_experts`

```
def rank_experts(importance: torch.Tensor, observed: torch.Tensor) -> list[int]
```

lines 65-74

Expert ids ordered from least to most important.

1. Sort by key `(unobserved?, importance, expert_id)` -> least important observed first, unobserved last.

`select_lowest`

```
def select_lowest(importance: torch.Tensor, observed: torch.Tensor, num_to_select: int,
allow_unobserved: bool=False) -> list[int]
```

lines 77-96 · raises ValueError

Pick the `num_to_select` least important experts of one layer.

1. Candidates = ranking, filtered to observed experts unless allowed.
2. If unobserved are allowed, re-sort so they come first (importance treated as -1).
3. Error if fewer candidates than requested or if all experts would be removed; return the chosen ids sorted.

Errors raised

Function	Exception	Message (f-string fields in braces)
`select_lowest`	ValueError	Cannot select {num_to_select} experts: only {len(candidates)} eligible (observed={int(observed.sum())}, allow_unobserved={allow_unobserved})
`select_lowest`	ValueError	Refusing to replace every expert of a layer